

# Technical Report: File Encryption System

May 5, 2026

## Contents

|          |                                                                       |          |
|----------|-----------------------------------------------------------------------|----------|
| <b>1</b> | <b>Project Summary</b>                                                | <b>2</b> |
| <b>2</b> | <b>Command List and Usage</b>                                         | <b>2</b> |
| <b>3</b> | <b>Challenges Faced During the Project</b>                            | <b>4</b> |
| 3.1      | Duress Mode Without Revealing the Authentic Payload . . . . .         | 4        |
| 3.2      | Cross-Platform Path Handling in MSYS2 and Windows . . . . .           | 4        |
| 3.3      | Discord Webhook Alerts on Windows . . . . .                           | 4        |
| 3.4      | Timing Attacks in HMAC Comparison . . . . .                           | 4        |
| 3.5      | Streaming Large Files Without Memory Exhaustion or Integer Overflow . | 4        |
| <b>4</b> | <b>How the Challenges Were Addressed</b>                              | <b>5</b> |
| 4.1      | Duress Mode . . . . .                                                 | 5        |
| 4.2      | Cross-Platform Path Handling . . . . .                                | 5        |
| 4.3      | Discord Webhook Alerts on Windows . . . . .                           | 5        |
| 4.4      | Timing-Safe HMAC Comparison . . . . .                                 | 5        |
| 4.5      | Streaming Large Files . . . . .                                       | 5        |
| <b>5</b> | <b>Further Scope for Enhancement</b>                                  | <b>6</b> |
| 5.1      | Authenticated Encryption Mode . . . . .                               | 6        |
| 5.2      | Memory-Hard Key Derivation . . . . .                                  | 6        |
| 5.3      | Graphical or Terminal User Interface . . . . .                        | 6        |
| 5.4      | Hardened Memory Handling . . . . .                                    | 6        |
| 5.5      | Portable File Format Serialization . . . . .                          | 6        |
| 5.6      | Expanded Automated Testing . . . . .                                  | 6        |

# 1 Project Summary

The File Encryption System is a command-line application written in C that provides secure encryption and decryption of files and folders. It is designed for users who require local file confidentiality, tamper detection, and coercion resistance.

The system uses **AES-256-CBC** for symmetric encryption, **PBKDF2-SHA256** for password-based key derivation, and **HMAC-SHA256** for integrity verification. After a successful encryption, the original plaintext file is permanently removed using a 3-pass secure shredding routine.

The project is organized into distinct modules:

- **main.c** — CLI parsing and workflow orchestration.
- **crypto/** — Encryption, decryption, key derivation, and integrity checks.
- **vault/** — On-disk **.enc** file format and header management.
- **security/** — Account lockout and secure file shredding.
- **utils/** — Audit logging, memory utilities, and password scoring.
- **alerts/** — Discord webhook notifications for suspicious events.

Notable features include a **duress/decoy mode**, which allows two independent payloads (one real, one decoy) to be stored inside a single **.enc** file, each unlocked by a different password. This enables plausible deniability under coercion. Additional features include live progress bars during encryption and decryption, an audit log of all operations, account lockout after five failed attempts, and Discord webhook alerts for unauthorized access events.

The system is built for cross-platform use, with specific handling for MSYS2 and Windows path conventions alongside standard Unix environments.

## 2 Command List and Usage

All commands follow the general form:

```
./encrypt <mode> <target> --password <password> [options]
```

### Encrypt a File

```
./encrypt --encrypt <file> --password <password>
```

Encrypts the specified file to **.enc** format. Verifies file integrity via SHA-256, evaluates password strength, encrypts using AES-256-CBC, and securely shreds the original.

```
./encrypt --encrypt secret.txt --password mypass123  
# Output: secret.txt.enc (original securely deleted)
```

## Encrypt a Folder

```
./encrypt --encrypt <folder> --password <password>
```

Archives the folder to `.tar.gz`, encrypts the archive, and shreds both the temporary archive and the original folder.

```
./encrypt --encrypt my_media --password mypass123  
# Output: my_media.tar.gz.enc
```

## Decrypt a File or Archive

```
./encrypt --decrypt <file.enc> --password <password>
```

Decrypts the `.enc` file. If the output is a `.tar.gz`, it is automatically extracted. The encrypted input is securely deleted after success.

```
./encrypt --decrypt document.pdf.enc --password mypass123  
./encrypt --decrypt my_photos.tar.gz.enc --password mypass123
```

## Verify Integrity

```
./encrypt --verify <file.enc> --password <password>
```

Checks the vault header and HMAC to confirm the file has not been tampered with, without producing any plaintext output.

```
./encrypt --verify secret.txt.enc --password MyS3cur3P@ss!
```

## Change Password

```
./encrypt --change-password <file.enc> --password <old> --new-password <new>
```

Re-encrypts the file with a new password and new salts/IVs without writing plaintext to disk. Duress-mode files also require old and new decoy passwords:

```
./encrypt --change-password secret.txt.enc \  
--password old_real --new-password new_real \  
--decoy-password old_decoy --new-decoy-password new_decoy
```

## Encrypt with Duress Mode

```
./encrypt --encrypt <file> --password <real_password> \  
--decoy <decoy_file> --decoy-password <decoy_password>
```

Stores a real file and a decoy file inside one `.enc` container, each protected by an independent password. Providing the decoy password reveals only the harmless file, enabling plausible deniability.

## Shred a File

```
./encrypt --shred <file>
```

Permanently deletes a file using three overwrite passes.

## View Audit Log

```
./encrypt --log
```

Displays the `audit.log` file, which records all encrypt, decrypt, and shred operations with timestamps and success/failure status.

# 3 Challenges Faced During the Project

## 3.1 Duress Mode Without Revealing the Authentic Payload

Implementing coercion-resistant encryption required that both the real and decoy files be indistinguishable from the outside. A naive design that placed the real file first and the decoy file second would immediately reveal which payload was authentic to any attacker examining the file layout.

## 3.2 Cross-Platform Path Handling in MSYS2 and Windows

Paths arriving from MSYS2 shells could be formatted in MSYS-style (`/g/...`) or contain Windows backslashes and shell-quoted segments. These formats caused `fopen()` failures on Windows, breaking file open, encryption, decryption, and shredding operations.

## 3.3 Discord Webhook Alerts on Windows

Sending HTTP requests from a C program on Windows is more complex than on Unix because there is no built-in `curl` command available in all environments, and temp-file paths and quoting rules differ from standard Unix conventions.

## 3.4 Timing Attacks in HMAC Comparison

A naive byte-by-byte comparison of two HMAC values short-circuits on the first mismatch. This leaks partial-match information through measurable timing differences, which can be exploited to test password candidates incrementally.

## 3.5 Streaming Large Files Without Memory Exhaustion or Integer Overflow

Loading multi-hundred-megabyte files entirely into memory is impractical. Additionally, computing progress percentages using 32-bit integers overflows for large file sizes, producing incorrect progress output.

## 4 How the Challenges Were Addressed

### 4.1 Duress Mode

Each payload was given its own independent salt, initialization vector, derived key, ciphertext block, and HMAC. During decryption, `decrypt_file()` tries the real password first; if HMAC verification fails, it falls back to the decoy password. The function returns distinct status codes — `DECRYPT_REAL`, `DECRYPT_DECOY`, and `DECRYPT_FAILED` — allowing the caller to adjust post-decryption behavior (for example, suppressing automatic shredding of the container when the decoy password is used). Because both payloads are stored in a single `.enc` file with no external marker identifying which is real, an attacker cannot distinguish the two.

### 4.2 Cross-Platform Path Handling

A `normalize_path()` function was added to `main.c`. It detects MSYS-style `/x/...` paths and converts the drive letter prefix to Windows-style `X:/`, then replaces any surviving backslashes with forward slashes. Local normalization was also added inside `crypto/encrypt.c` for duress input paths and inside `security/shredder.c` for deletion paths. This ensures that paths from any shell format reach `fopen()` in a usable state.

### 4.3 Discord Webhook Alerts on Windows

The alerting module `alerts/discord.c` was implemented to write a formatted JSON payload to a known temp path (`C:\msys64\tmp`), then write a curl config file referencing the payload and the webhook URL, and finally invoke `curl.exe` to post the notification. The webhook URL is compiled in as a fallback but can be overridden at runtime via the `DISCORD_WEBHOOK_URL` environment variable, allowing deployment without recompilation.

### 4.4 Timing-Safe HMAC Comparison

The `integrity_verify()` function was rewritten to use a full-length XOR accumulation loop with a `volatile` accumulator variable. This prevents the compiler from optimizing away comparisons on early mismatch and ensures that the loop always runs to completion, making the comparison constant-time at the algorithmic level regardless of where the first differing byte appears.

### 4.5 Streaming Large Files

Encryption and decryption process data in 65 KB chunks (`CHUNK_SIZE = 65536`), keeping memory usage constant regardless of file size. Progress counters were changed to `long long` to prevent overflow. On Windows, `_filelengthi64()` is used to obtain the full 64-bit file size, ensuring accurate percentage reporting for files larger than 2 GB.

## 5 Further Scope for Enhancement

### 5.1 Authenticated Encryption Mode

The current design uses AES-256-CBC with a separate HMAC. Migrating to **AES-256-GCM** would provide built-in authenticated encryption (AEAD), eliminating the need for an external HMAC step and reducing the risk of implementation errors in the authenticate-then-decrypt ordering.

### 5.2 Memory-Hard Key Derivation

PBKDF2-SHA256 is functional but does not resist GPU or ASIC-based offline dictionary attacks as well as modern alternatives. Replacing it with **Argon2id** would significantly increase the cost of brute-force password guessing by requiring large amounts of memory in addition to computational work.

### 5.3 Graphical or Terminal User Interface

The system is currently command-line only, which creates friction for non-technical users, particularly for multi-step workflows such as duress-mode encryption. A terminal UI (TUI) or lightweight GUI would make the tool more accessible without sacrificing the existing CLI.

### 5.4 Hardened Memory Handling

Passwords and derived keys currently reside in standard heap memory during processing. Page-locking those allocations (via `mlock` on Unix or `VirtualLock` on Windows) and zeroing them as soon as they are no longer needed would reduce exposure through swap files or memory dumps.

### 5.5 Portable File Format Serialization

The vault header currently uses raw C struct serialization, which is sensitive to compiler padding and ABI differences across platforms and toolchains. Replacing this with explicit packed serialization (manually writing each field at a defined byte offset) would make the file format portable and stable across compiler versions.

### 5.6 Expanded Automated Testing

Current tests cover the main cryptographic and file-handling paths. Automated tests for webhook delivery, lockout thresholds under concurrent access, and password-change correctness for duress-mode files would increase confidence in correctness during future refactoring.